Trabalho Prático 1

Diogo Alexandre Alves da Silva nº31504

GitHub: **https://github.com/disilva021P/ISI_01_2627**

Ano Letivo 26/27

Docente: Óscar Ribeiro

Integração de Sistemas da Informação

Conteúdo

Enquadramento 4
Problema 5
Descrição do cenário 5
O que se pretende resolver? 5
Objetivos 6
Estratégia utilizada 6
Visão geral da arquitetura 6
Tecnologias utilizadas 6
Decisões de arquitetura 7
Transformações 8
Transformacao_LojaA: TeleCompra (CSV) 8
Explicação 8
Transformacao_LojaB: AmeriMobile (XML) 9
Transformacao_API: especificações técnicas 10
Transformacao_API_Cambio: taxa de câmbio 11
Transformacao_Verificar_Erros: consolidação de erros 11
Transformacao_Reconciliar: reconciliação e carga 12
Transformacao_Saidas: agregações e exportações 14
Expressões regulares utilizadas 16
Jobs 16
Job_Principal 16
Lógica do fluxo 18
Parâmetros 18
Serviços de suporte, Docker e dashboard 19
Execução com Docker 19
Notificações por e-mail 20
Node-RED: dashboard e flow 21
Mosquitto e MQTT 23
Segurança e credenciais 24
Vídeo com demonstração 25
Conclusão e trabalhos futuros 26
Referências bibliográficas 27

Índice de Figuras

Figura 1 – Transformacao_LojaA: leitura do CSV, validação em duas fases, normalização, agregação de duplicados e ramo de erros 8
Figura 2 – Transformacao_LojaB: leitura do XML, correção do preço, validação, normalização, agregação e ordenação 9
Figura 3 – Transformacao_API: pedido REST às especificações, validação, normalização e remoção de duplicados 10
Figura 4 – Transformacao_API_Cambio: obtenção da taxa USD→EUR e definição da variável TAXA_USD_EUR 11
Figura 5 – Transformacao_Verificar_Erros: consolidação dos erros das três origens num CSV e paragem se existirem erros 12
Figura 6 – Transformacao_Reconciliar: junção das lojas e das especificações, conversão para EUR e carga em tb_produtos 13
Figura 7 – Transformacao_Saidas: agregações por marca e por loja, anonimização MD5 e exportação em CSV, JSON, YAML e SQL 14
Figura 8 – Pasta output/ com os ficheiros de staging, as agregações, as exportações anonimizadas e o resumo da execução 16
Figura 9 – Job_Principal: sequência dos passos, ramo de falha para Email erro e Abortar job e ramo dos erros de dados 17
Figura 10 – E-mail “Email erros dados” com a lista consolidada de registos inválidos em anexo 21
Figura 11 – E-mail “Email sucesso” com as exportações geradas em anexo 21
Figura 12 – Flow Node-RED: leitura dos CSV a cada 30 s, gráficos e tabelas do dashboard e publicação MQTT nos tópicos etl/* 22
Figura 13 – Dashboard Node-RED: taxa usada, produtos reconciliados, gráficos por marca, resumo por loja e último evento MQTT 23

Enquadramento

No âmbito da unidade curricular de Integração de Sistemas de Informação (ISI), do curso de Licenciatura em Engenharia de Sistemas Informáticos da Escola Superior de Tecnologia do IPCA (ano letivo 2026/2027), apresenta-se o relatório relativo ao Trabalho Prático I. O projeto procura abordar a integração de sistemas ao nível dos dados através da implementação e exploração de processos de ETL (Extract, Transform, Load).

Problema

Descrição do cenário

O cenário escolhido é imaginário mas realista. Um retalhista de telemóveis recebe o stock de duas lojas parceiras, a TeleCompra (Loja A) e a AmeriMobile (Loja B). Cada uma exporta os seus dados num formato diferente e com qualidade diferente, e o retalhista precisa de um catálogo único e fiável para decidir o que vender.

Origem	Formato	Conteúdo	Problemas típicos
TeleCompra (Loja A)	CSV (loja_a.csv)	id, marca, modelo, quantidade	Marca ou modelo nulos, quantidades nulas ou negativas, nomes de modelo inconsistentes, duplicados
AmeriMobile (Loja B)	XML (loja_b.xml)	codigoProduto, nomeMarca, dispositivo, sistemaOperativo, anoLancamento, unidades, precoUnitario, moeda, estado	Preço com vírgula decimal, preço em USD, anos de lançamento inválidos, nulos e duplicados
API de especificações	JSON via REST (localhost:8000)	marca, modelo, ram_gb, armazenamento_gb, bateria_mah	Registos sem marca ou modelo, repetidos
API de câmbio (Frankfurter)	JSON via REST	Taxa USD para EUR	Serviço externo que pode falhar, exigindo valor por defeito

O que se pretende resolver?

Pretende-se construir um pipeline automatizado que:

- recolhe os dados das várias origens heterogéneas (ficheiros CSV e XML, duas APIs REST em JSON);
- valida, limpa e normaliza os dados, incluindo expressões regulares, e isola os registos inválidos sem interromper o processamento dos válidos;
- reconcilia os catálogos das duas lojas numa única tabela de produtos, identificando os modelos comuns e os exclusivos de cada loja;
- converte os preços de USD para EUR com a taxa de câmbio do dia e enriquece cada produto com as especificações técnicas vindas da API;
- persiste o resultado numa base de dados MySQL e gera exportações anonimizadas em CSV, JSON, YAML e SQL;
- notifica por e-mail o sucesso, os registos com erro e as falhas do pipeline, e alimenta um dashboard Node-RED.

Objetivos

Este trabalho tem como principais objetivos consolidar os conceitos fundamentais de integração de sistemas ao nível dos dados, aplicando-os através do Pentaho Kettle no desenvolvimento de transformações e de um job de orquestração. Para tal, procura-se demonstrar na prática as três fases do processo ETL (Extract, Transform, Load) a partir de fontes com formatos e origens diversificadas, articulando ainda a solução desenvolvida com tecnologias complementares como Docker, MySQL, Node-RED e Mosquitto (MQTT).

Estratégia utilizada

Visão geral da arquitetura

O pipeline segue a estrutura das três fases exigida no enunciado. Os dados são extraídos de quatro origens, tratados por sete transformações Kettle e orquestrados por um único job. Entre as transformações usam-se ficheiros XML de staging (stg_*.xml), o que separa cada passo permitindo inspecionar os dados intermédios e facilita repetir apenas parte do processo.

A arquitetura escolhida visou preencher todos os requisitos de mais valia apontados pelo docente: expressões regulares, serialização XML/JSON/YAML, job com e-mail e logs, acesso a APIs, joins e agregações, operações em base de dados e dashboard.

Fase	Objetivo	Componentes
Fase 1 (E): Recolha	Ler origens e formatos diversos	CSV Input (loja A), Get data from XML (loja B), Data Grid + REST client + JSON input (API de especificações e API de câmbio)
Fase 2 (T): Transformação	Limpar, validar, normalizar, cruzar e agregar	Janino (Java Expression), Replace in string e Regex Evaluation, Filter Rows, Group By, Sort Rows, Merge Join, Calculator, Modified Java Script Value
Fase 3 (L): Carga e exportação	Persistir, exportar, notificar e visualizar	Table Output (MySQL), Text File Output (CSV, YAML, SQL), JSON Output, XML Output, e-mail, Node-RED Dashboard, publicação MQTT (Mosquitto)

Tecnologias utilizadas

- Pentaho Kettle (PDI): Ferramenta principal de ETL, utilizada para a criação das transformações (.ktr) e do job de orquestração (.kjb).
- MySQL 8.0 (Docker): Base de dados de destino (bd_lojab), acessível através da porta 3307 no host.
- API REST Local (Docker): Serviço em container (porta 8000) responsável por fornecer as especificações técnicas dos modelos.

- API Frankfurter: Serviço externo consultado para obter a taxa de câmbio atualizada de USD para EUR.
- Node-RED (Docker): Plataforma utilizada para construir o dashboard de visualização (em http://localhost:1880/ui). Lê os ficheiros CSV gerados na pasta output/ e publica os resultados em tópicos MQTT.
- Eclipse Mosquitto (Docker): Broker MQTT (porta 1883) onde o Node-RED publica os resultados do ETL e a que qualquer cliente se pode subscrever (tópicos etl/...).

- Docker Compose: Ferramenta responsável por orquestrar todos os serviços em containers, permitindo a execução do Kettle através do comando *docker compose run --rm kettle*.

Decisões de arquitetura

- Chave de reconciliação normalizada. Os produtos das duas lojas são cruzados pela chave MARCA|MODELO, depois de normalizada (maiúsculas, sem a marca repetida no modelo, sem parênteses, sem espaços, "+" convertido em PLUS).
- Tratamento de erros sem paragem. Os registos inválidos são desviados para ficheiros XML de erros, consolidados num CSV, enviados por e-mail, e o pipeline continua com os registos válidos.
- Câmbio como variável global. A taxa é lida uma vez e guardada na variável JVM TAXA_USD_EUR, com valor por defeito 0.90 caso a API falhe, e reutilizada na conversão.
- Anonimização. As exportações substituem os identificadores originais por hashes MD5.
- Parametrização e logs. O job define parâmetros para o SMTP e os e-mails (SMTP_SERVER, SMTP_PORT, EMAIL_USER, EMAIL_PASSWORD, EMAIL_ORIGEM e EMAIL_DESTINO), passados a todas as transformações, e cada transformação escreve um log próprio, com data e hora, na pasta logs/.
- Publicação em tempo quase real. Os resultados agregados são também publicados em tópicos MQTT (QoS 1, retain), para que outros sistemas os consumam sem ler ficheiros.
- Segredos fora do código. A palavra-passe da base de dados e os dados SMTP (incluindo a palavra-passe de aplicação do Gmail) não estão nos ficheiros .ktr/.kjb: chegam como parâmetros do job, preenchidos a partir de variáveis de ambiente (ficheiro .env, excluído do Git).

Transformações

Transformacao_LojaA: TeleCompra (CSV)

Lê o ficheiro loja_a.csv, valida-o em duas fases (regras de negócio e Regex), normaliza o texto, soma as quantidades de duplicados e produz stg_loja_a.xml.

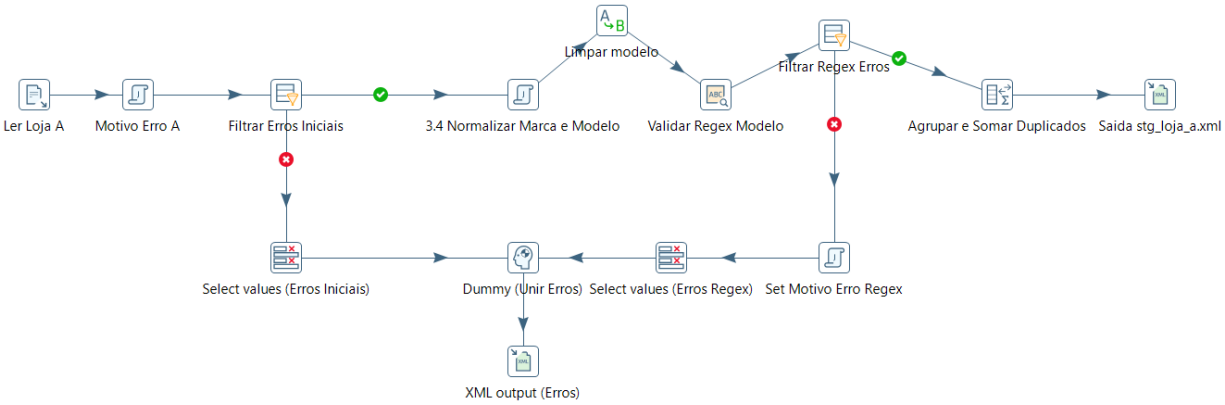


Figura 1 – Transformacao_LojaA: leitura do CSV, validação em duas fases, normalização, agregação de duplicados e ramo de erros

Explicação

Passo	Tipo	Função
Ler Loja A	CSV File Input	Lê data/loja_a.csv (separador ",", UTF-8, cabeçalho): id, marca, modelo, quantidade
Motivo Erro A	Java Expression	Marca nula, Modelo nulo, Quantidade nula, Quantidade negativa ou OK
Filtrar Erros Iniciais	Filter Rows	Separa registos válidos dos inválidos
Normalizar Marca e Modelo	Java Expression	Cria nomeMarca e dispositivo normalizados
Limpar modelo	Replace in string	Três substituições por Regex
Validar Regex Modelo	Regex Evaluation	Valida o dispositivo com ^[a-zA-Z0-9\s\.\-\+]+$ e cria modelo_valido
Filtrar Regex Erros	Filter Rows	Os modelos inválidos seguem para o ramo de erros
Set Motivo Erro Regex	Java Expression	Define o motivo "Modelo com formato inválido (Regex)"
Select values (Erros ...) e Dummy (Unir Erros)	Select Values, Dummy	Reúne os dois tipos de erro (id e motivo)

Passo	Tipo	Função
XML output (Erros)	XML Output	Escreve output/erros/erros_transformação_lojaA (com data)
Agrupar e Somar Duplicados	Memory Group By	Agrupa por marca e dispositivo; primeiro id como codigoProduto; soma de quantidade em unidades
Saida stg_loja_a.xml	XML Output	Escreve output/stg_loja_a.xml (codigoProduto, nomeMarca, dispositivo, unidades)

Transformacao_LojaB: AmeriMobile (XML)

Lê o ficheiro loja_b.xml, corrige o formato do preço, valida, normaliza, agrega duplicados, ordena e produz stg_loja_b.xml.

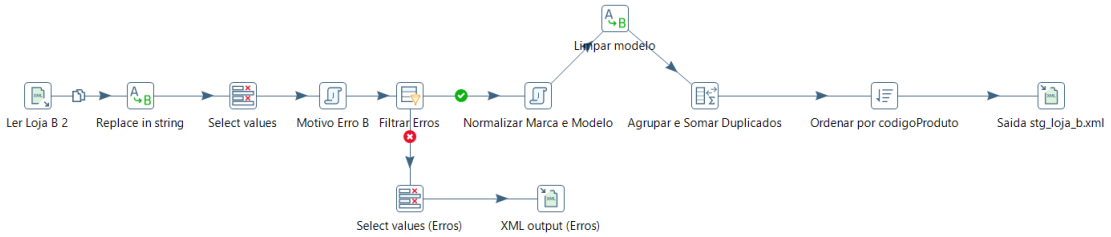


Figura 2 – Transformacao_LojaB: leitura do XML, correção do preço, validação, normalização, agregação e ordenação

Passo	Tipo	Função
Ler Loja B 2	Get data from XML	Lê data/loja_b.xml com o ciclo /produtos/produto
Replace in string	Replace in string	Troca a vírgula decimal por ponto no precoUnitario
Select values	Select Values	Converte precoUnitario para número (máscara #.##)
Motivo Erro B	Java Expression	Marca nula, Modelo nulo, Quantidade nula, Quantidade negativa, Preço negativo ou OK
Filtrar Erros	Filter Rows	Separa registos válidos dos inválidos
Normalizar Marca e Modelo	Java Expression	Cria nomeMarca_clean e dispositivo_clean
Limpar modelo	Replace in string	Três substituições por Regex sobre dispositivo_clean
Agrupar e Somar Duplicados	Memory Group By	Agrupa por marca e dispositivo limpos; soma as unidades

Passo	Tipo	Função
Ordenar por codigoProduto	Sort Rows	Ordenação ascendente
Saida stg_loja_b.xml	XML Output	Escreve output/stg_loja_b.xml (9 campos, incluindo preço, moeda e estado)
Select values (Erros) e XML output (Erros)	Select Values, XML Output	Escreve output/erros/erros_transformação_lojaB.xml (com data)

Transformacao_API: especificações técnicas

Consome a API REST local de especificações, valida e normaliza os registos e gera o ficheiro de staging stg_especificacoes.xml.

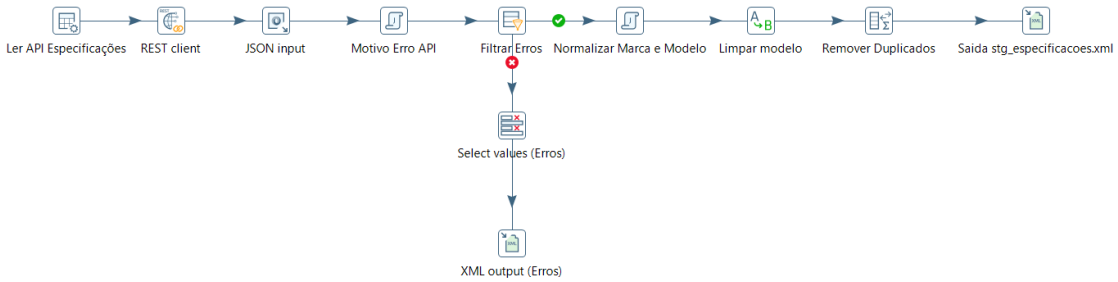


Figura 3 – Transformacao_API: pedido REST às especificações, validação, normalização e remoção de duplicados

Passo	Tipo	Função
Ler API Especificações	Data Grid	Fornece o URL http://localhost:8000/api/especificacoes
REST client	REST client	Pedido GET ao URL, devolvendo o JSON em texto
JSON input	JSON Input	Extrai marca, modelo, ram_gb, armazenamento_gb e bateria_mah com JSONPath ($.[*].campo)
Motivo Erro API	Java Expression	Calcula o motivo: “Marca nula”, “Modelo nulo” ou “OK”
Filtrar Erros	Filter Rows	Encaminha os registos com motivo OK para a normalização e os restantes para o ramo de erros
Normalizar Marca e Modelo	Java Expression	nomeMarca em maiúsculas e sem espaços; dispositivo sem a marca repetida
Limpar modelo	Replace in string	Três expressões regulares (ver secção Expressões regulares utilizadas)

Passo	Tipo	Função
Remover Duplicados	Memory Group By	Agrupa por nomeMarca e dispositivo e fica com o primeiro valor de cada especificação
Saida stg_especificacoes.xml	XML Output	Escreve output/stg_especificacoes.xml
Select values (Erros) e XML output (Erros)	Select Values, XML Output	Guarda os registos inválidos em output/erros/erros_transformação_api (com data)

Transformacao_API_Cambio: taxa de câmbio

Obtém a taxa USD para EUR no serviço Frankfurter e guarda-a numa variável global para ser usada na reconciliação. O job executa-a antes de qualquer conversão de preços.



Figura 4 – Transformacao_API_Cambio: obtenção da taxa USD→EUR e definição da variável TAXA_USD_EUR

Passo	Tipo	Função
URL API Cambio	Data Grid	URL https://api.frankfurter.app/latest?base=USD&symbols=EUR
REST Client Frankfurter	REST client	Pedido GET ao serviço externo
Extrair Taxa EUR	JSON Input	Lê $.rates.EUR como número (taxa_eur)
Definir Variavel TAXA_USD_EUR	Set Variables	Guarda a taxa na variável JVM TAXA_USD_EUR, com valor por defeito 0.90

Como a variável é do tipo JVM, fica disponível para todas as transformações seguintes do mesmo job. O valor por defeito dá resiliência a falhas do serviço externo.

Transformacao_Verificar_Erros: consolidação de erros

Lê os ficheiros de erros gerados pelas três transformações anteriores, junta-os num CSV único com a origem, a referência e o motivo, e sinaliza se existem erros.

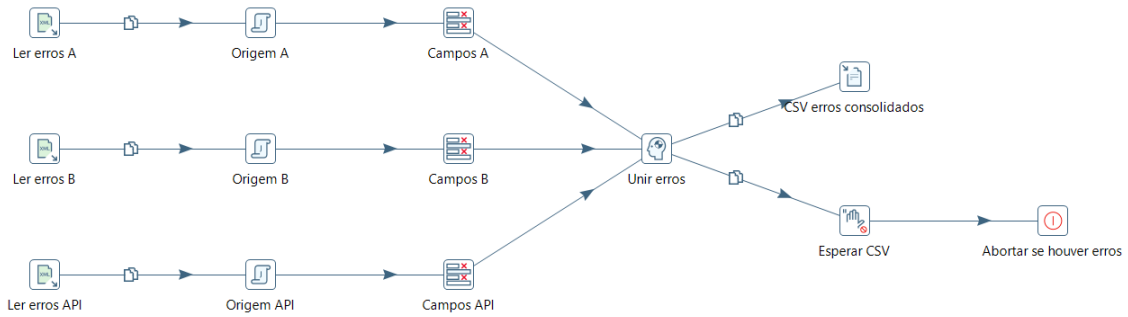


Figura 5 – Transformacao_Verificar_Erros: consolidação dos erros das três origens num CSV e paragem se existirem erros

Passo	Tipo	Função
Ler erros A / B / API	Get data from XML	Lê os XML de output/erros, filtrando pelo nome (.*lojaA.*, .*lojaB.*, .*_api.*)
Origem A / B / API	Java Expression	Acrescenta origem (TeleCompra, AmeriMobile, API) e referência (id, código ou marca e modelo)
Campos A / B / API	Select Values	Uniformiza as colunas: origem, referencia e motivo
Unir erros	Dummy	Junta os três fluxos
CSV erros consolidados	Text File Output	Escreve output/erros/erros_consolidados.csv
Esperar CSV	Block until steps finish	Garante que o CSV está escrito antes do passo seguinte
Abortar se houver erros	Abort	Termina com erro se chegar alguma linha (limiar 0)

Transformacao_Reconciliar: reconciliação e carga

Esta transformação é o núcleo do projeto. Une os catálogos das duas lojas, converte preços, junta as especificações e carrega a tabela tb_produtos.

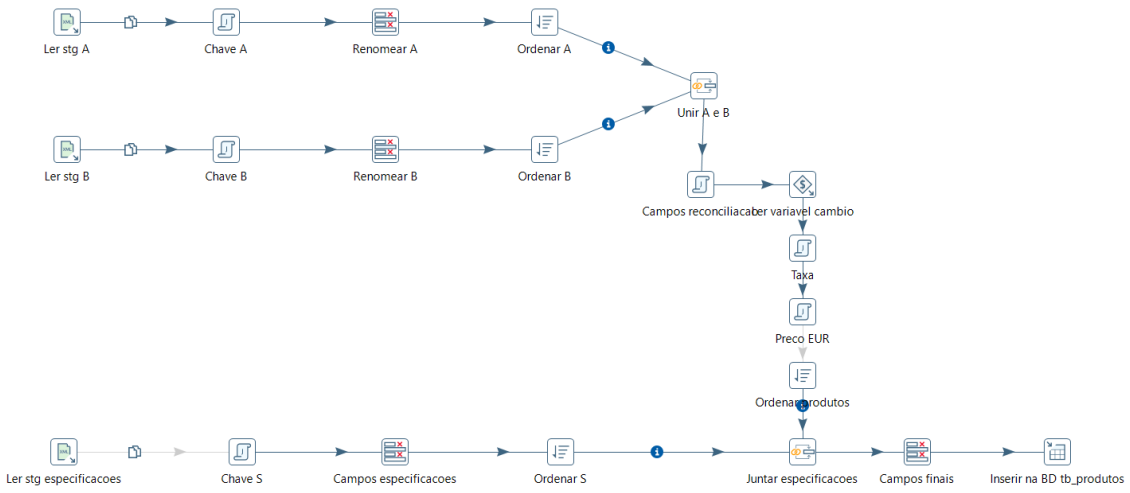


Figura 6 – Transformacao_Reconciliar: junção das lojas e das especificações, conversão para EUR e carga em tb_produtos

Passo	Tipo	Função
Ler stg A / stg B / stg especificacoes	Get data from XML	Lê os três ficheiros de staging
Chave A / B / S	Java Expression	Constrói a chave nomeMarca|dispositivo
Renomear A / B e Campos especificacoes	Select Values	Prefixa os campos (id_a, unidades_a, id_b, unidades_b, preco_usd, estado_b) para evitar colisões; nas especificações seleciona chave_s, ram_gb, armazenamento_gb e bateria_mah
Ordenar A / B / S	Sort Rows	Ordena por chave, requisito do Merge Join
Unir A e B	Merge Join (FULL OUTER)	Mantém todos os produtos de ambas as lojas
Campos reconciliacao	Java Expression	Calcula chave, tipo, marca e modelo finais, unidades_total, estado_produto, ano_valido e ano_aviso
Ler variavel cambio e Taxa	Get Variables, Java Expression	Lê ${TAXA_USD_EUR}; se vazia ou inválida usa 0.90
Preco EUR	Java Expression	preco_eur = preco_usd × taxa, arredondado a 2 casas
Ordenar produtos	Sort Rows	Ordena por chave
Juntar especificacoes	Merge Join (LEFT OUTER)	Associa ram_gb, armazenamento_gb e bateria_mah
Campos finais	Select Values	Seleciona as 19 colunas finais
Inserir na BD tb_produtos	Table Output	Carrega em MySQL, truncando a tabela, com commit de 1000 e batch

As regras de reconciliação mais relevantes são:

- **tipo:** "Exata" se o produto existe nas duas lojas; "So TeleCompra" ou "So AmeriMobile" caso contrário.
- **unidades_total e estado_produto:** soma das unidades das duas lojas; "Esgotado" se for zero, "Disponivel" caso contrário.
- **ano_valido e ano_aviso:** anos de lançamento fora do intervalo 1990–2026 ficam nulos e geram o aviso "Ano invalido".
- **preco_eur:** conversão do preço em USD com a taxa obtida na transformação API_CAMBIO.

Transformacao_Saidas: agregações e exportações

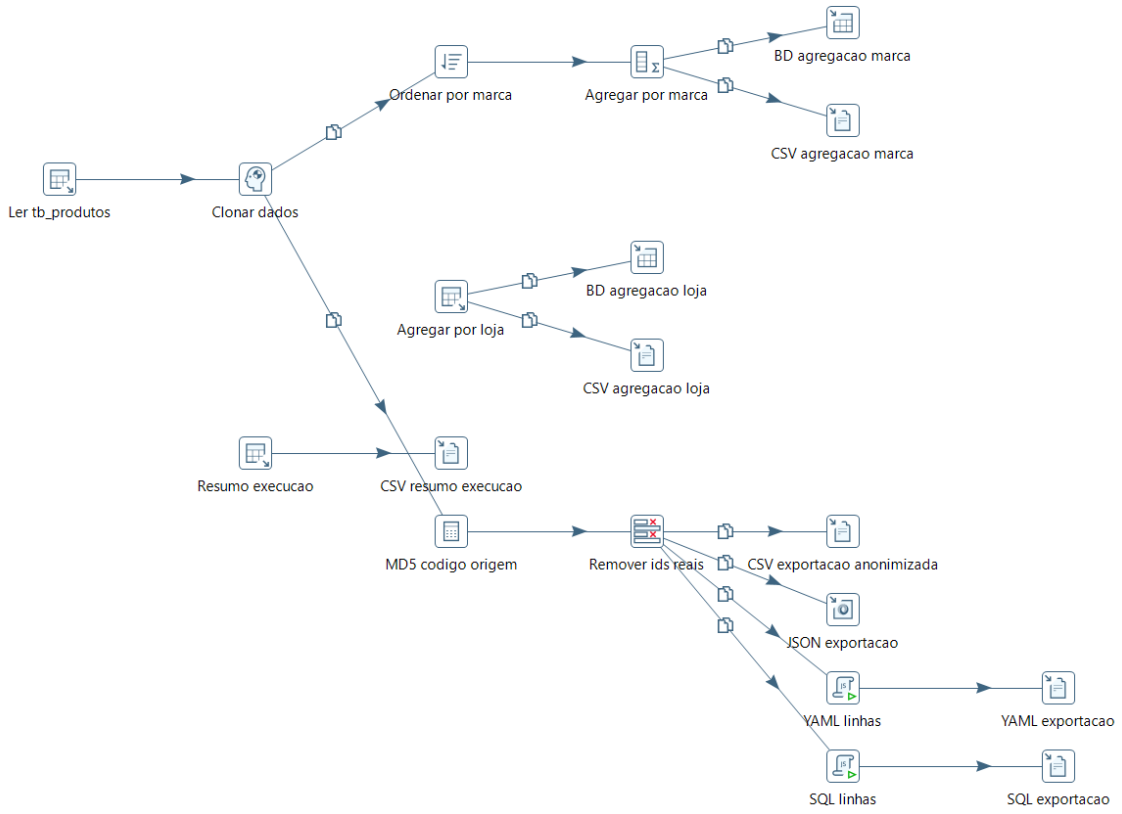


Figura 7 – Transformacao_Saidas: agregações por marca e por loja, anonimização MD5 e exportação em CSV, JSON, YAML e SQL

Passo	Tipo	Função
Ler tb_produtos	Table Input	SELECT * FROM tb_produtos
Clonar dados	Dummy (cópias de linhas)	Distribui as linhas pelos vários ramos

Passo	Tipo	Função
Ordenar por marca e Agregar por marca	Sort Rows, Group By	Por marca: n_modelos (COUNT), unidades_total (SUM) e preco_medio_eur (AVERAGE)
BD agregacao marca e CSV agregacao marca	Table Output, Text File Output	Escreve tb_agg_marca e output/agg_marca.csv
Agregar por loja	Table Input	SQL com UNION ALL: n_modelos, unidades e preço médio por loja
BD agregacao loja e CSV agregacao loja	Table Output, Text File Output	Escreve tb_agg_loja e output/agg_loja.csv
MD5 codigo origem	Calculator	MD5 de id_a e id_b (id_a_md5, id_b_md5)
Remover ids reais	Select Values	Retira os identificadores originais, ficando só os hashes
CSV exportacao anonimizada	Text File Output	output/export_produtos_anonimizado.csv
JSON exportacao	JSON Output	output/export_produtos_anonimizado.json
YAML linhas e YAML exportacao	Modified Java Script Value, Text File Output	Serializa cada linha em YAML e escreve o ficheiro .yaml
SQL linhas e SQL exportacao	Modified Java Script Value, Text File Output	Gera CREATE TABLE IF NOT EXISTS e INSERTs em .sql
Resumo execucao e CSV resumo execucao	Table Input, Text File Output	Taxa usada, total de produtos, com e sem especificações, data e hora: output/resumo_execucao.csv

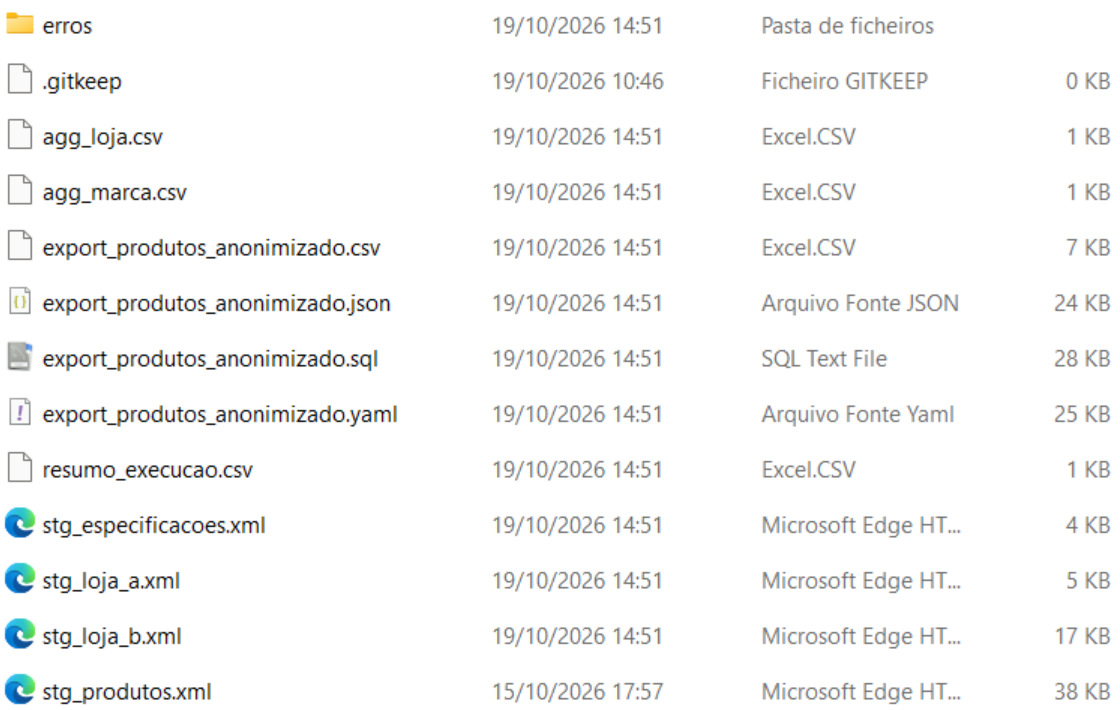


Figura 8 – Pasta output/ com os ficheiros de staging, as agregações, as exportações anonimizadas e o resumo da execução

Expressões regulares utilizadas

Expressão	Substituição	Objetivo
\+	PLUS	Converter o símbolo "+" em texto (ex.: Galaxy S23+ passa a S23PLUS)
\s*\(.*?\)	(vazio)	Remover informação entre parênteses
\s+	(vazio)	Remover todos os espaços, normalizando o modelo para a chave
^[a-zA-Z0-9\s\.\-\+]+$	validação	Aceitar apenas modelos com caracteres alfanuméricos, espaço, ponto, hífen e "+"

Jobs

Job_Principal

O Job_Principal orquestra todo o pipeline. Corre as transformações em sequência, controla o fluxo consoante o sucesso ou a falha de cada passo e envia as notificações por e-mail.

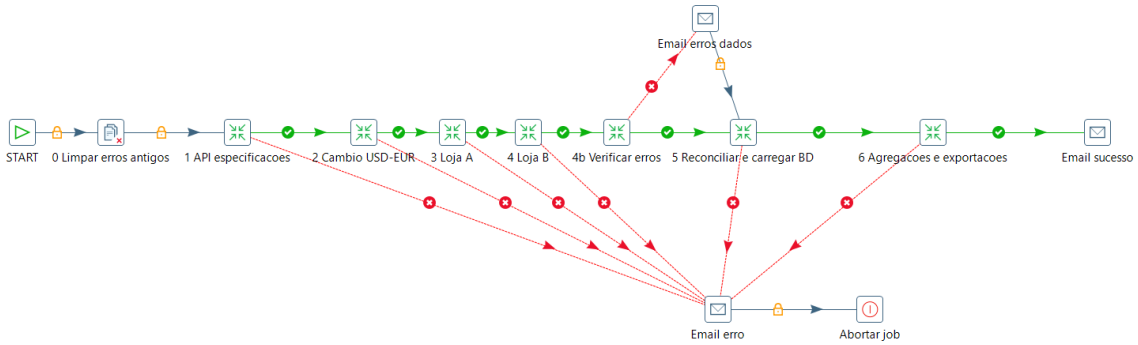


Figura 9 – Job_Principal: sequência dos passos, ramo de falha para Email erro e Abortar job e ramo dos erros de dados

Entrada	Tipo	Função
START	Start	Ponto de partida do job
0 Limpar erros antigos	Delete files	Apaga os .xml e .csv de output/erros de execuções anteriores
1 API especificacoes	Transformation	Executa Transformacao_API
2 Cambio USD-EUR	Transformation	Executa Transformacao_API_Cambio e define TAXA_USD_EUR
3 Loja A	Transformation	Executa Transformacao_LojaA
4 Loja B	Transformation	Executa Transformacao_LojaB
4b Verificar erros	Transformation	Executa Transformacao_Verificar_Erros
Email erros dados	Mail	Envia o CSV consolidado dos registos inválidos
5 Reconciliar e carregar BD	Transformation	Executa Transformacao_Reconciliar
6 Agregacoes e exportacoes	Transformation	Executa Transformacao_Saidas
Email sucesso	Mail	Envia o resumo com as exportações em anexo
Email erro	Mail	Notifica falha de um passo técnico
Abortar job	Abort	Termina o job com a mensagem “Pipeline falhou – ver logs”

Cada entrada de transformação escreve um ficheiro de log em logs/ (nível Basic), com data e hora no nome. Os parâmetros do job são passados a todas as transformações.

Lógica do fluxo

- A cadeia principal é START → 0 → 1 → 2 → 3 → 4 → 4b → 5 → 6 → Email sucesso, com cada passo a avançar apenas em caso de sucesso.
- Se qualquer uma das transformações 1 a 6 falhar, o job segue pelo ramo de falha para Email erro e depois para Abortar job.
- Depois da Loja B, o passo 4b decide: se não há registos inválidos avança diretamente para a reconciliação; se há, a Transformacao_Verificar_Erros termina com erro por desenho, o job envia o Email erros dados (com o CSV consolidado em anexo) e continua incondicionalmente para o passo 5 com os dados válidos.
- O Email sucesso inclui os ficheiros gerados pelo pipeline como anexos.

Parâmetros

Parâmetro	Valor por defeito	Descrição
SMTP_SERVER	smtp.gmail.com	Servidor SMTP
SMTP_PORT	587	Porta SMTP (587 com STARTTLS)
EMAIL_USER	origem@exemplo.pt	Utilizador de autenticação
EMAIL_PASSWORD	(vazio; definida em execução)	Palavra-passe de aplicação
EMAIL_ORIGEM	origem@exemplo.pt	Remetente
EMAIL_DESTINO	destino@exemplo.pt	Destinatário
DB_ROOT_PASSWORD	(vazio; definida em execução)	Palavra-passe do MySQL

Em execução no Docker, estes valores vêm do ficheiro .env através do docker-compose.yml (serviço kettle, secção environment) e são passados ao Kitchen com -param:. Os três passos de e-mail do job usam estes parâmetros (por exemplo ${SMTP_SERVER} e ${EMAIL_PASSWORD}), por isso nenhuma credencial fica escrita nos ficheiros. Fora do Docker (por exemplo no Spoon) aplicam-se os valores por defeito acima e é preciso preencher EMAIL_PASSWORD e DB_ROOT_PASSWORD em Run options → Parameters.

Serviços de suporte, Docker e dashboard

O ambiente é definido num único docker-compose.yml com cinco serviços. O serviço kettle corre sob demanda (perfil "job") e só arranca depois de a base de dados estar saudável e a API iniciada.

Serviço	Imagem / origem	Porta	Função
db	mysql:8.0	3307	Base de dados bd_lojab (script de arranque em docker/db-init)
api	./api	8000	API REST de especificações
nodered	./docker/nodered	1880	Dashboard sobre output/ (montada só de leitura) e publicação dos resultados em MQTT
kettle	./docker/kettle	—	Executa o Job_Principal (docker compose run --rm kettle)
mosquitto	eclipse-mosquitto:2	1883	Broker MQTT (configuração em docker/mosquitto/mosquitto.conf; dados persistidos no volume mqttdata)

Execução com Docker

Todo o ambiente está descrito num único docker-compose.yml, o que torna o projeto reprodutível: quem clonar o repositório só precisa do Docker e de um ficheiro .env (com DB_ROOT_PASSWORD e os dados SMTP: SMTP_SERVER, SMTP_PORT, EMAIL_USER, EMAIL_ORIGEM, EMAIL_DESTINO e EMAIL_PASSWORD) para ter os cinco serviços disponíveis. Os pontos principais da configuração são os seguintes.

- **Dependências e healthcheck.** A base de dados declara um healthcheck (mysqladmin ping). O serviço kettle só arranca quando a BD está saudável (service_healthy) e a API foi iniciada (service_started); sem isto o job tentaria ligar-se a uma BD ainda a inicializar.
- **Perfil job.** O kettle tem profiles: ["job"], por isso docker compose up levanta apenas os serviços de suporte e o ETL é lançado a pedido com docker compose run --rm kettle.
- **Volumes.** dbdata guarda os dados do MySQL; ./docker/db-init é montado em /docker-entrypoint-initdb.d e cria as tabelas apenas na primeira criação do volume; mqttdata guarda as mensagens retidas do Mosquitto; ./output é montado só de leitura no Node-RED; o kettle monta a raiz do projeto em /project (acesso a .ktr/.kjb, data/, output/ e logs/).
- **Portas.** 3307→3306 (MySQL), 8000 (API), 1883 (MQTT) e 1880 (Node-RED). A BD usa a 3307 no anfitrião para não colidir com um MySQL local.

A imagem do Kettle (docker/kettle) parte de eclipse-temurin:11-jre, copia a instalação do PDI para /opt/pdi e usa um entrypoint.sh que prepara a execução:

- copia os .ktr/.kjb para /work, para não alterar os originais do projeto;
- cria ligações simbólicas /work/data, /work/output e /work/logs para as pastas do projeto, de modo a que os caminhos ${Internal.Entry.Current.Directory}/output/... funcionem dentro do container;
- substitui, só nas cópias, localhost por db e http://localhost:8000 por http://api:8000. Dentro de um container localhost é o próprio container, pelo que se usam os nomes dos serviços, resolvidos pelo DNS interno do Docker. Os mesmos ficheiros funcionam no Spoon e no Docker;
- passa DB_ROOT_PASSWORD ao Kettle como parâmetro do job, que resolve ${DB_ROOT_PASSWORD} na ligação à base de dados;
- executa o kitchen.sh sobre o Job_Principal.kjb, passando com -param: o servidor SMTP, a porta, o utilizador, o remetente, o destinatário, a palavra-passe de aplicação e a palavra-passe da base de dados.

Sequência de execução usada na demonstração:

```
docker compose up -d db api mosquitto nodered
docker compose run --rm kettle
docker compose down -v      # repõe o ambiente (apaga os volumes)
```

Depois de o job terminar: e-mails na caixa de correio do destinatário, dashboard em http://localhost:1880/ui, base de dados em 127.0.0.1:3307 (schema bd_lojab) e ficheiros em output/. Se a pasta docker/db-init for alterada, é preciso recriar o volume (docker compose down -v) para o script voltar a correr.

Notificações por e-mail

O job envia os e-mails por SMTP (Gmail) para o endereço de destino configurado. Existem três mensagens: sucesso (com as exportações em anexo), registos de dados inválidos (com o CSV consolidado em anexo) e falha técnica de um passo. As figuras seguintes mostram duas delas, tal como foram recebidas.



Figura 10 – E-mail "Email erros dados" com a lista consolidada de registos inválidos em anexo

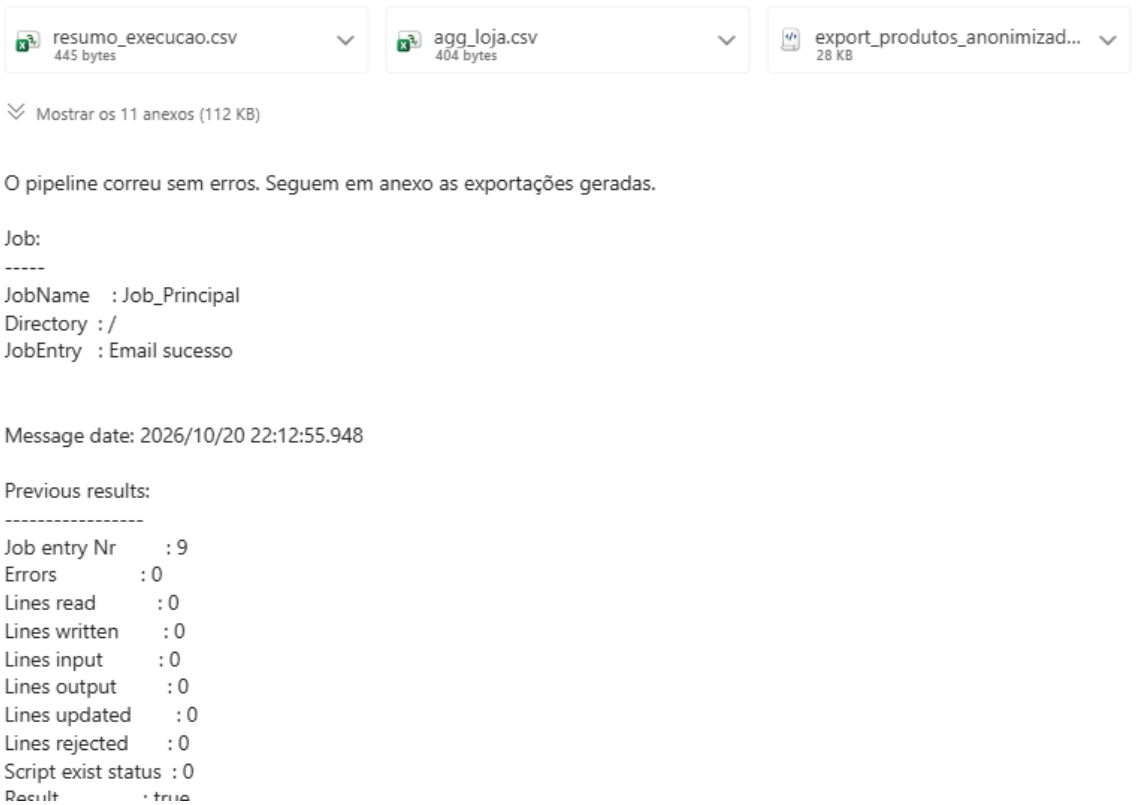


Figura 11 – E-mail "Email sucesso" com as exportações geradas em anexo

Node-RED: dashboard e flow

O Node-RED consome os resultados do ETL e apresenta-os num dashboard. O flow (figura seguinte) tem o seguinte funcionamento:

- **Disparo.** Um nó inject ("refresh 30s") executa ao arrancar (após 2 s) e depois a cada 30 segundos.
- **Leitura.** Três nós file in leem /output/agg_marca.csv, /output/agg_loja.csv e /output/resumo_execucao.csv, montados só de leitura. Cada um é seguido de um nó csv (separador ";", primeira linha como cabeçalho) que converte o texto em objetos.
- **Preparação.** Três funções preparam os dados: Marcas gera duas séries (nº de modelos e preço médio em EUR por marca); Lojas formata a tabela por loja; Resumo extrai a taxa de câmbio, o total de produtos e a contagem com e sem especificações. Na função das marcas, a primeira mensagem de cada série limpa o gráfico, evitando acumular dados antigos a cada atualização.
- **Apresentação.** Dois gráficos de barras (ui_chart), uma tabela por loja e os indicadores de resumo (ui_template), no separador "Produtos", grupo "Resultados do ETL".

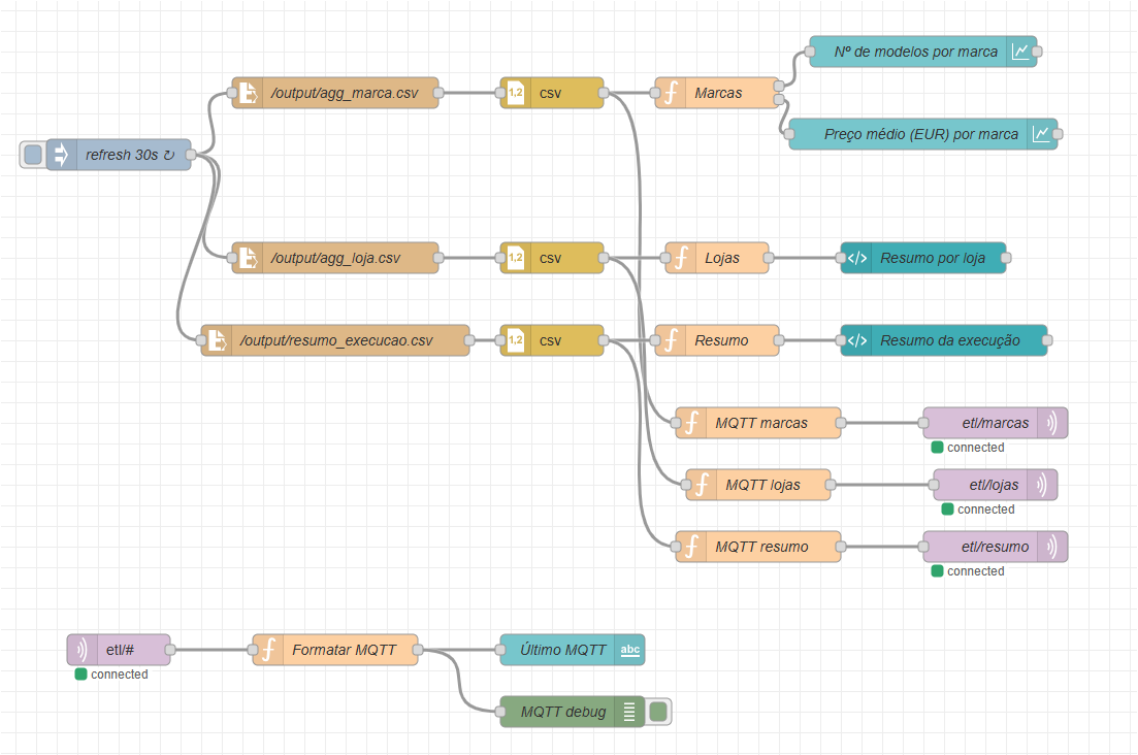


*Figura 12 – Flow Node-RED: leitura dos CSV a cada 30 s, gráficos e tabelas do dashboard e publicação MQTT nos tópicos etl/**

Como o dashboard lê os ficheiros periodicamente, atualiza-se sozinho quando o job volta a correr, sem reiniciar o Node-RED. Antes da primeira execução do job, os gráficos mostram "sem dados (corre o Job)". Na execução apresentada, a taxa USD→EUR foi 0,8917 e foram reconciliados 74 produtos, 16 deles com especificações técnicas.

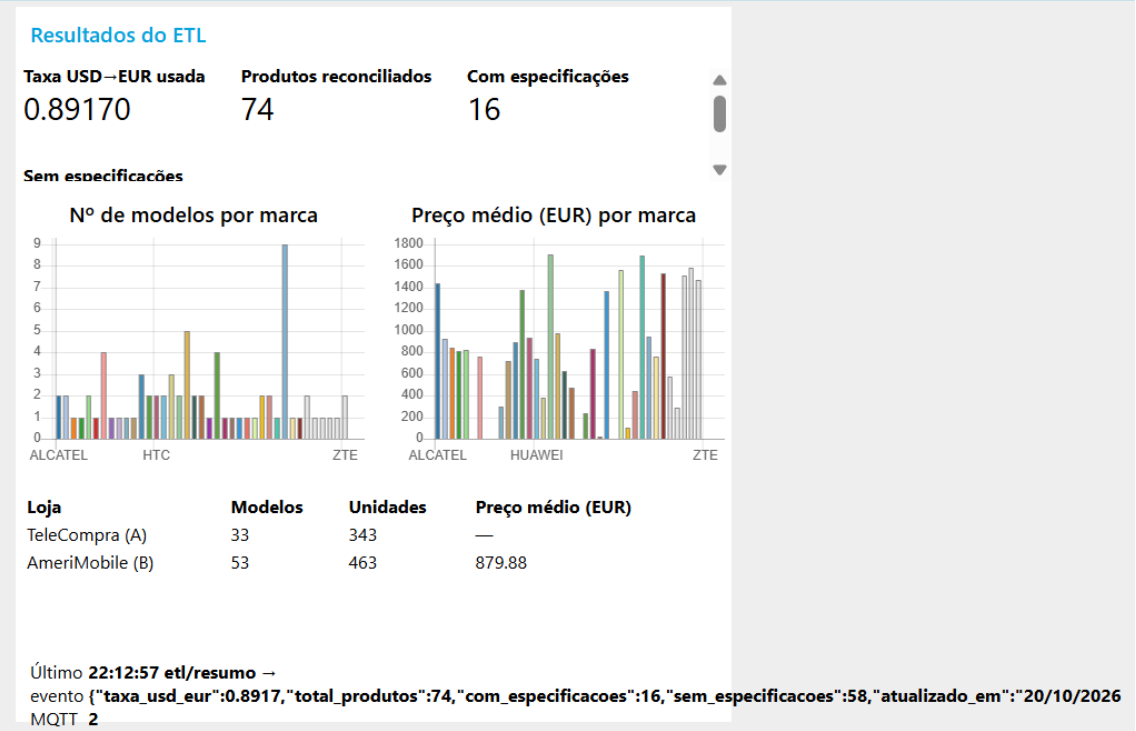


Figura 13 – Dashboard Node-RED: taxa usada, produtos reconciliados, gráficos por marca, resumo por loja e último evento MQTT

Mosquitto e MQTT

O Eclipse Mosquitto é o broker MQTT do projeto. O MQTT segue o paradigma publicar/subscrever: quem produz dados publica num tópico e quem os quer subscreve-o, sem se conhecerem. Aqui, o Node-RED publica os resultados do ETL e qualquer outro cliente (Home Assistant, Grafana, um script) pode consumi-los. A configuração (docker/mosquitto/mosquitto.conf) define o listener 1883, aceita ligações anónimas (adequado a um ambiente de testes local) e ativa a persistência em /mosquitto/data, ligada ao volume mqttdata, para que as mensagens retidas sobrevivam a reinícios.

Tópico	Conteúdo	QoS / retain	Publicado por
etl/marcas	Lista JSON com marca_final, n_modelos, unidades_total e preco_medio_eur (de agg_marca.csv)	1 / sim	Node-RED
etl/lojas	Lista JSON com loja, n_modelos, unidades_total e preco_medio_eur (de agg_loja.csv)	1 / sim	Node-RED
etl/resumo	Objeto JSON com taxa_usd_eur, total_produtos, com_especificacoes, sem_especificacoes e atualizado_em	1 / sim	Node-RED
etl/nodered/status	Texto “online” (birth) ou “offline” (last will)	1 / sim	Node-RED / broker

- **QoS 1 e retain.** Com QoS 1 cada mensagem é entregue pelo menos uma vez. Com retain, o broker guarda a última mensagem de cada tópico e entrega-a de imediato a um subscritor novo, sem esperar pelo ciclo seguinte de 30 s.
- **Birth e last will.** Ao ligar, o Node-RED publica "online" em etl/nodered/status; se a ligação cair de forma inesperada, o broker publica "offline". Outros serviços ficam assim a saber se o dashboard está ativo.
- **Subscrição.** Um nó mqtt in subscreve etl/# e alimenta o indicador "Último evento MQTT" e a consola de debug, o que demonstra o ciclo completo publicar → broker → subscrever. O mesmo se pode verificar fora do Node-RED:

```
docker compose exec mosquitto mosquitto_sub -t "etl/#" -v
```

Limitação: o MQTT é usado aqui como canal de saída. Em produção, o broker deveria ter autenticação por utilizador, listas de controlo de acesso e TLS (porta 8883); esta evolução está indicada nos trabalhos futuros.

Segurança e credenciais

Os ficheiros .ktr e .kjb não contêm credenciais: a palavra-passe da base de dados e os dados SMTP chegam como parâmetros do job (SMTP_SERVER, SMTP_PORT, EMAIL_USER, EMAIL_ORIGEM, EMAIL_DESTINO, EMAIL_PASSWORD e DB_ROOT_PASSWORD), passados pelo entrypoint do contentor a partir do ficheiro .env (excluído do repositório pelo .gitignore, com um .env.example como modelo). Para o Gmail usa-se uma palavra-passe de aplicação e não a da conta. Esta opção foi tomada porque a cifragem "Encrypted" do Kettle é apenas ofuscação reversível, pelo que guardar nela uma palavra-passe real num repositório público seria inseguro.

Vídeo com demonstração

O vídeo apresenta a execução completa do pipeline: arranque dos serviços, execução do job, resultados na base de dados, ficheiros exportados, e-mails recebidos e dashboard.

Link para o vídeo



Conclusão e trabalhos futuros

O trabalho cumpriu o objetivo de construir um pipeline ETL completo com o Pentaho Kettle. Foram desenvolvidas sete transformações e um job de orquestração que integram quatro origens heterogéneas (CSV, XML e duas APIs REST em JSON), validam e normalizam os dados com expressões regulares, reconciliam dois catálogos com Merge Join, convertem os preços com uma taxa de câmbio obtida em tempo de execução, persistem o resultado em MySQL e o exportam em CSV, JSON, YAML e SQL com anonimização por MD5. O tratamento de erros sem paragem, as notificações por e-mail e o ambiente Docker tornam o processo robusto e reprodutível, e o Node-RED com o Mosquitto permite visualizar e distribuir os resultados em tempo quase real.

Principais dificuldades:

- O Merge Join exige entradas ordenadas pela mesma chave e o modo FULL OUTER para manter os produtos exclusivos de cada loja.
- O Table Output não cria tabelas nem mapeia colunas: o esquema da base de dados tem de coincidir com os 19 campos enviados, e o script de arranque só corre quando o volume é criado.
- Dentro dos containers, localhost deixa de apontar para os outros serviços, o que obrigou à substituição feita no entrypoint.
- O passo Abort, usado para sinalizar erros de dados, obriga a distinguir esses casos das falhas técnicas do job.

Limitações conhecidas:

- O MQTT é usado apenas como saída; os dados de entrada vêm de ficheiros e APIs.
- O dashboard atualiza por leitura periódica dos CSV (30 s) e não por eventos.
- A coluna moeda da Loja B é lida, mas todos os preços são tratados como USD.
- O Mosquitto não tem autenticação nem TLS.

Trabalhos futuros:

- Usar o MQTT também como origem de dados (por exemplo, atualizações de stock simuladas no Node-RED, publicadas em loja/stock e consumidas pelo Kettle).
- Exportar também para Apache Parquet e enviar os ficheiros por SFTP.
- Agendar o job (cron ou Apache Airflow) e ligar o Grafana ou o Home Assistant aos tópicos etl/#.
- Proteger o Mosquitto com autenticação, ACL e TLS, e acrescentar testes automáticos com conjuntos de dados de teste.

Referências bibliográficas

- Pentaho Data Integration (Kettle), documentação oficial: https://docs.hitachivantara.com/r/en-us/pentaho-data-integration
- Node-RED, documentação: https://nodered.org/docs/
- MQTT, especificação: https://mqtt.org/mqtt-specification/
- Docker Compose, documentação: https://docs.docker.com/compose/
- Frankfurter, API de taxas de câmbio: https://frankfurter.dev/
- Repositório do projeto: https://github.com/disilva021P/ISI_01_2627